

DataForge

Standard Operating Procedure

Sales-Operations Platform — Lead Scraping, Organization & Performance Tracking
Version 1.0 · Desktop Application (Electron) · Last updated: 20 August 2026

Table of Contents

1. Overview & Architecture
2. Roles & Access Control
3. Getting Started (Install, Login, Tray)
4. Navigation (Sidebar, Tabs, Breadcrumb, Lite Mode)
5. Dashboard
6. Leads Management
7. Scraping (Website & Auto Keywords)
8. Reports & Performance
9. GHL Integration (Calls, Appointments, Sync)
10. Marketing Suite
11. Forger — AI Assistant
12. Administration (Users & Fleet)
13. Settings & Feature Toggles
14. How the Code Works (Technical)
15. Desktop Build & Deployment
16. Troubleshooting
17. Appendix — Route Reference

1. Overview & Architecture

DataForge is a desktop sales-operations application that scrapes business leads, organizes them into a searchable database, and tracks agent performance — with an optional AI assistant and a live integration to GoHighLevel (GHL).

What it does

- **Scrapes leads** from the web (by website domain or by keyword + location).
- **Organizes leads** into Categories → Subcategories → Folders, with quality scoring and de-duplication.
- **Tracks performance** — per-agent call/appointment/lead metrics, leaderboards, commissions.
- **Integrates with GHL** — syncs calls and appointments from the connected sub-account.
- **Assists via "Forger"** — an in-app AI helper (boss/admin) for finding data and running tasks.

Technical architecture

- **Desktop app (Electron):** installs on each device and runs a local Next.js server. Closing the window keeps it running in the system tray; a single-instance lock prevents duplicate app states.
- **Database:** a shared hosted PostgreSQL (Supabase) — all installs read/write the same data.
- **Scraper:** a dedicated headless Chromium (Playwright) is downloaded automatically on the first scrape (one time, per device) — it is not bundled in the installer, keeping the download small.

Key idea: the app is one window into a shared database. What one user changes (leads, settings, folders) is visible to everyone, subject to their role.

2. Roles & Access Control

Every user has one role that determines what they can see and do. Roles are assigned in **Admin** → **Users**.

Role	Access
boss	Full access, plus Settings , Feature Toggles , and How It Works . The only role that can change global configuration.
admin	Full access <i>except</i> boss-only Settings and Feature Toggles.
team_lead / sales_rep	Marketing department: Dashboard, Marketing suite, Kanban, Calendar, Reports (view), Profile.
lead_specialist	Leads department: Leads and Scraping — but only for the categories/keywords explicitly granted to them (see Administration).

Access is enforced on the server, not just hidden in the UI. A disabled feature (via Feature Toggles) is hidden from the sidebar and its page is blocked for everyone.

3. Getting Started

3.1 Installing & launching

1. Run the DataForge installer on the device. Launch **DataForge**.
2. The window opens maximized. Sign in with your email and password.
3. Closing the window does **not** quit the app — it minimizes to the **system tray** and keeps running. Re-opening focuses the existing instance (no duplicate state).
4. To fully quit, right-click the tray icon and choose Quit.

3.2 Signing in

- Use the email/password provided by your boss/admin.
- Change your display name, photo, and password anytime in **My Profile** ([/profile](#)).
- Signing out returns you to the sign-in screen immediately.

Shared reports exception: pages under [/share](#) (shared reports) are public — anyone with the link can view them without signing in.

4. Navigation

- **Left sidebar** reaches every page you have access to. It can be collapsed; the collapsed state is remembered across restarts.
- **Browser-style tabs:** double-click a sidebar item (or click its hover "open in new tab" icon) to open it in a tab. Tabs persist across app restarts.
- **Breadcrumb:** the header shows the trail of pages visited in the current tab. Use the back arrow or **Backspace** to go back. It shows up to 5 crumbs; older ones fade and the trail scrolls.
- **Lite Mode:** the feather icon in the top bar toggles Lite Mode — fewer animations, lighter on RAM. It is per-user and remembered. Recommended on lower-spec devices.
- **Theme:** toggle light/dark from the top bar.

5. Dashboard

Route: `/dashboard` . The landing page after login.

- Org-wide metrics: total leads, quality distribution, leads by industry/category.
- Charts and recent-leads table for a quick pulse of activity.
- For marketing roles it also surfaces the leaderboard and period metrics (Today / Week / Month).

6. Leads Management

Route: </leads> . The central lead database.

6.1 Structure

Leads are organized in a hierarchy:

Categories → Subcategories → Folders → Leads

- **Category** (a.k.a. Industry): top-level grouping. Opening a category shows its subcategories and folders.
- **Subcategory**: optional grouping inside a category.
- **Folder**: holds the actual leads. A category shows an **"Ungrouped"** bucket for folders that have no subcategory.
- Leads saved to a category *without* choosing a specific folder land in that category's automatic **"Ungrouped"** folder.
- There is also a global **"Unfiled"** list for leads that belong to no folder at all (</leads/list?folder=unfiled>).

6.2 Folder cards & export indicator

- Each folder card shows its lead count, owner, and created/updated dates.
- An **export progress bar** (battery-style) sits by the "Click to view leads →" hint, showing the **percentage of that folder's leads already exported**. If nothing has been exported, no bar is shown.

6.3 Lead quality & de-duplication

- Every lead is **quality-scored 0–100%** by completeness: business name, phone, email, address, website, and category.
- **De-duplication**: incoming leads are matched by phone / business name and discarded if already present.

6.4 Viewing & exporting

- Open a folder to search, filter, and export leads. Filters include has-phone, has-email, has-website, quality, etc.
- **All Leads (list view)**: </leads/list> — a flat, filterable table across everything.
- **Add Lead manually**: </leads/new> .
- **CSV export**: from a folder, export **all** filtered leads or only the **not-yet-exported ("unexported")** ones. Exported leads get an export-date stamp, which feeds the folder's export percentage bar.

Scraped leads and GHL-webhook leads are stored separately. The Leads board shows **scraped** leads.

7. Scraping

Route: `/scraping` . Two methods (tabs):

7.1 Scrape a Website

Tab: `/scraping?tab=domain` . Crawl a single domain for contact details (emails, phones). Enter the domain and run; results are captured as leads. Email grabbing loads each page (running its JavaScript) and scans the rendered HTML for thorough contact extraction.

7.2 Auto Keywords

Tab: `/scraping?tab=keywords` . Saved **keyword + location** pairs, organized in category folders.

- Each keyword belongs to a **category folder**; the leads it finds save into that category.
- **Run now** — scrapes the keyword once.
- **Auto run** — repeats scraping until you stop it. (There is **no** scheduling/cron.)
- **City rotation** and **extra-keyword variations** help surface fresh leads; the location picker sorts/cycles cities by population.
- **Adaptive backoff** throttles keywords that keep returning duplicates, so effort concentrates where new leads exist.
- Sort keyword folders alphabetically or by lead count; the choice is remembered across restarts.

Keyword access can be scoped per user: a lead_specialist only sees the categories/keywords a boss/admin has granted them (see Administration).

8. Reports & Performance

Route: </reports> (boss/admin). A per-agent performance matrix.

- **Heatmaps** for Leads, Appointment Performance, and Call Performance — darker cells mean stronger relative performance.
- **Call Performance "Today" column is clickable** — a calendar opens so you can view calls for any chosen date.
- **KPI strip:** active agents, appointments this month, calls (24h / week / month), average connect rate.
- **Shareable public link:** generate a link that anyone can open (no login). On the desktop app the link uses the production domain so external recipients can view it.
- **Timezone-aware:** "Today"/month boundaries are computed in the boss-configured timezone (Settings → set to match GHL) so day counts line up.

9. GHL Integration (GoHighLevel)

DataForge connects to one GHL sub-account (configured in Settings → Integrations: API key, Location ID).

9.1 What syncs

- **Calls** — imported from GHL via the API and attributed to the agent who made each call.
- **Appointments** — synced from GHL calendars and attributed to the booking agent.
- Agents are matched to GHL by their **GHL User ID** (set per user in Admin → Users).

9.2 Call counts — source of truth

- Call data comes from the **GHL API sync** — one row per real call. The GHL webhooks no longer write call rows (they over-counted).
- Trigger a sync from **Marketing** → **Sync GHL Calls**. The circular-arrow (↻) button does a full **"Rebuild from GHL"**: it deletes existing call logs and re-imports fresh.
- **Timezone**: set the app timezone (Settings) to match GHL so per-day counts align.

Note on GHL's dashboard "Agent Report": GHL's dashboard call number is a proprietary, filtered metric that GHL does **not** expose over its API. DataForge counts actual call activity from the GHL conversations API; this can legitimately differ from GHL's dashboard figure. There is no API that returns the dashboard number.

9.3 Requirements

- The GHL API token must be a Private Integration Token with the **Conversations** read scopes (`conversations.readonly` , `conversations/message.readonly`).

10. Marketing Suite

Feature	Route	Purpose
Overview	/marketing	Marketing metrics + leaderboard.
Notes	/marketing/notes	Shared team notes.
Scripts	/marketing/scripts	Call scripts.
My Leads	/marketing/my-leads	Leads assigned to the current agent.
Badges	/marketing/manage/badges	Achievement badge management.
Challenges	/marketing/manage/tasks	Marketing tasks / challenges.
Commissions	/marketing/manage/commissions	Commission management (boss/admin).
My Commissions	/my-commissions	An agent's own commissions.
Balloon Pop	/balloons	Gamified team activity.

Each of these is a **toggleable feature** — a boss can hide any of them for the whole team (Settings → Features).

11. Forger — AI Assistant

Forger is an in-app AI helper (boss/admin only) that helps you **use** DataForge — find pages, look up data, export CSVs, and (on desktop) start/stop scrapers. It is a product helper, not a coding tool, and it can only **read** the database (plus CSV export and scraper control).

11.1 Opening Forger

- Click the floating **"Ask Forger"** button (bottom-right). It opens a resizable chat panel (drag the left edge; toggle full-screen).
- Chat history is saved; you can delete a conversation or clear all.

11.2 Setup (boss)

- Forger needs an **Anthropic API key**, set once in **Settings** → **Forger**. With a key set, Forger works for everyone on every device with no per-user setup.
- Alternatively, on a device that has Claude Code installed and logged in, Forger can use that — but a single central API key is the recommended, zero-setup path.
- A **request size guard** (max tokens, configurable in Settings → Forger) blocks oversized prompts to control cost.

11.3 What Forger can do

- Answer "where is X" and "how do I..." with clickable in-app links.
- Look up overview stats, list categories/keywords, search leads, export leads to CSV.
- On the desktop app only: start a keyword scrape or stop all scrapers.
- It will refuse to write/modify data or discuss source code.

12. Administration

12.1 Users — [/admin/users](#)

- Create users, set their **role**, reset details.
- Set each agent's **GHL User ID** so GHL calls/appointments attribute correctly.
- **Keyword access grants:** scope which categories/keywords a **lead_specialist** can see and work.

12.2 Fleet — [/admin/fleet](#)

- See connected devices and running scrapes in real time.
- A device drops from the fleet immediately when its app closes.

13. Settings & Feature Toggles

Route: `/settings` (boss for global config; other roles see Profile/Appearance/Security).

Tab	Contents
General	Company name, scraping defaults (max leads, concurrency, run limits), global pause / boost.
Leads	Lead quality thresholds (good/medium).
Integrations	GHL API key, sub-account key, Location ID, webhook/inbound settings, commission currency, timezone (searchable — set to match GHL).
Features	Turn app modules on/off for the whole team (see below). Boss only.
Forger	Anthropic API key, model, max request tokens, and the Forger enable toggle.
Maintenance	Housekeeping / data operations.
Profile	Display name, nickname, photo, password.
Appearance	Theme and accent.
Security	Account security settings.

13.1 Feature Toggles

Boss-only. Each toggle hides a module from the sidebar and blocks its page for the whole team: **Marketing Overview, Notes, Scripts, Badges, Challenges, Commissions, Balloon Pop, Leads, Scraping.**

Settings save automatically when you change a field or flip a toggle; saves auto-retry once on a transient database hiccup.

14. How the Code Works (Technical)

This section is for developers/maintainers. The active codebase is the `dataforge-app-lite` project; `dataforge-app` is a frozen backup and must not be edited.

14.1 Technology stack

Layer	Technology
UI & server	Next.js 16 (App Router, React Server Components, Server Actions, <code>output: "standalone"</code>)
Database	PostgreSQL (hosted on Supabase) via Prisma 7 with the pg adapter
Auth	NextAuth (Credentials provider; middleware enforces login, server code enforces roles)
Desktop shell	Electron (wraps the Next standalone server; system tray; single-instance lock)
Scraper	Playwright + Chromium (downloaded on first scrape into a per-user path; headless page load, renders JS, extracts contacts)
AI (Forger)	Anthropic API SDK (primary) or the Claude Agent SDK via local Claude Code (fallback)
Real-time	Socket.IO (custom <code>server.ts</code>) for fleet/live updates

14.2 Project structure

Path	Purpose
<code>src/app/</code>	Routes (App Router). <code>(app)/</code> = authenticated pages, <code>(auth)/</code> = sign-in, <code>api/</code> = API routes & webhooks. <code>error.tsx</code> / <code>global-error.tsx</code> render friendly error screens.
<code>src/lib/</code>	Server-side services & helpers — e.g. <code>ghl/</code> (sync + client), <code>folders/</code> , <code>industry/</code> , <code>leads/</code> , <code>reports/</code> , <code>marketing/</code> , <code>forger/</code> , <code>settings/</code> , <code>utils/timezone.ts</code> , <code>prisma.ts</code> .
<code>src/actions/</code>	Server Actions — the write path called from the client (settings, folders, keyword access, auth, etc.).
<code>src/components/</code>	React components (client & shared UI). <code>ui/</code> = base-ui/shadcn primitives.
<code>prisma/schema.prisma</code>	Database schema (models: User, Lead, Folder, Industry/Subcategory, CallLog, GhlAppointment, AppSettings, Forger*, etc.).
<code>electron/</code>	Desktop shell: <code>main.js</code> (boots the server + window + tray), <code>assemble.mjs</code> , <code>after-pack.js</code> .
<code>server.ts</code>	Custom Node server (Next + Socket.IO) — the entry the desktop app runs.

14.3 Request & data flow

1. A **page** (server component in `src/app/(app)/.../page.tsx`) runs on the server, checks the session/role, and calls a **service** in `src/lib/...` .

- The service uses the **Prisma client** (`src/lib/prisma.ts`) to query PostgreSQL and returns typed data to the page, which renders HTML.
- Mutations** (saving settings, toggling features, creating folders) go through **Server Actions** in `src/actions/...`, which re-check the role (`requireRole`), write via Prisma, then `revalidatePath`. Transient DB blips auto-retry via `withDbRetry`.
- Errors** bubble to the nearest `error.tsx`; uncaught ones hit `global-error.tsx` — both show a friendly "Something went wrong" screen instead of a raw stack.

14.4 How the desktop app runs

- `electron/main.js` acquires a **single-instance lock**, starts the bundled **Next standalone server** (from `resources/standalone`) as a child process, then opens a window pointed at `http://localhost:3000`.
- Closing the window hides to the **tray**; the server keeps running so re-opening is instant.
- On the first scrape the app **downloads Chromium** into a persistent per-user folder (`PLAYWRIGHT_BROWSERS_PATH` under the app's user-data dir) and reuses it thereafter.
- The DB connection string lives in `.env.local`, which is copied into the bundle at assemble time.

14.5 Key subsystems (where the logic lives)

Feature	Code
GHL call/appointment sync	<code>src/lib/ghl/sync.ts</code> + <code>src/lib/ghl/client.ts</code> (one CallLog per real call message, attributed by the caller's GHL userId).
GHL webhooks	<code>src/app/api/ghl/{inbound,outbound}-call/route.ts</code> (CallLog writes disabled — API sync is the source of truth).
Sync trigger + rebuild	<code>src/app/api/ghl/sync-calls/route.ts</code> (supports a <code>wipe</code> full-rebuild) + <code>SyncGhlCallsButton.tsx</code> .
Folder export % bar	<code>src/lib/folders/service.ts</code> (<code>attachExportedCounts</code>) + <code>FolderExportBar.tsx</code> .
Timezone-aware dates	<code>src/lib/utils/timezone.ts</code> (used by reports & marketing services).
Forger AI	<code>src/app/api/forger/chat/route.ts</code> (API-key-first), <code>src/lib/forger/*</code> (tools, agent, knowledge).

15. Desktop Build & Deployment

The desktop app is built from the active project (`dataforge-app-lite`) with **three commands, in this exact order**:

Step 1 — Build the web bundle

```
npm run desktop:build-web
```

- Runs `prisma generate` then `next build --webpack` with `BUILD_TARGET=desktop`.
- Produces `.next/standalone/` — a self-contained Node server (`server.js`) with traced dependencies.
- **Note:** Next intentionally does *not* include static assets, `public/`, or env files in the standalone output — Step 2 adds them.

Step 2 — Assemble the bundle

```
npm run desktop:assemble
```

 (runs `electron/assemble.mjs`)

- Copies `.next/static/` → `.next/standalone/.next/static/` (the JS/CSS chunks the pages load).
- Copies `public/` and `.env.local` (the DB connection string) into the bundle.
- Ensures the Playwright install machinery (`playwright` / `playwright-core`) is present. Chromium itself is **not** bundled — it is downloaded on the first scrape (see §14.4), which keeps the installer ~60% smaller.

Step 3 — Package the installer

```
npm run desktop:pack
```

 (runs `electron-builder --win`)

- Builds the Windows NSIS installer into `dist/`.
- The `afterPack` hook (`electron/after-pack.js`) copies the assembled `.next/standalone` into the app's `resources/standalone` (a plain recursive copy, so dot-folders like `.next` and `.env.local` are included).

First time only: `npm run desktop:browsers` (installs the Chromium that gets bundled). **All-in-one:** `npm run desktop:dist` runs `browsers` → `build-web` → `assemble` → `pack`.

Critical ordering: `assemble` must run **after** `build-web` and **before** `pack`. Re-running `build-web` after `assemble` regenerates `.next/standalone` and wipes the copied static/env. Before packing, verify both exist:

- `.next/standalone/.next/static/chunks` — else the packaged app is blank / all JS & CSS 404.
- `.next/standalone/.env.local` — else pages throw a server-side database error.

Disk space: since Chromium is no longer bundled, the app bundle is now much smaller (~450 MB). Packaging still needs a few GB free for the app copy + compressed installer + temp. The error `can't write ... bytes to output` means the drive is out of space — free space and retry.

First-run browser download: the first time a device runs a scrape, DataForge downloads Chromium once (~1–2 min, needs internet) into a per-user folder; subsequent scrapes reuse it. The install is idempotent, so it is a fast no-op thereafter.

16. Troubleshooting

Symptom	Cause & Fix
"Something went wrong" screen / pages fail to load	Usually a temporary database connection drop (hosted Supabase). Click Try again . If persistent, the network/DB region connection is unstable — stabilize the connection or move the DB region.
Settings toggles won't save	Same transient DB drop — the save auto-retries once; try again after a moment. The toggle logic itself is fine.
Packaged app is blank / JS & CSS 404	The build was packaged without static assets — <code>assemble</code> didn't run (or <code>build-web</code> ran after it). Rebuild in order and verify <code>.next/standalone/.next/static</code> .
Packaged app: server-side exception on load	The bundle is missing <code>.env.local</code> (no DB connection). Re-run <code>assemble</code> and verify the file before packing.
Installer fails: "can't write N bytes to output"	Disk full. Free several GB and re-run <code>desktop:pack</code> .
Forger says it can't reach Claude Code	No AI credential set. Boss adds an Anthropic API key in Settings → Forger (works for everyone, no per-device setup).
GHL calls not importing (401)	The GHL token lacks the Conversations read scopes. Recreate/edit the Private Integration Token with <code>conversations.readonly</code> and <code>conversations/message.readonly</code> .
Call counts differ from GHL dashboard	Expected — GHL's dashboard number is a private, filtered metric not available via API. DataForge counts real call activity from the GHL API.
Duplicate app state on reopen	Prevented by the single-instance lock. If odd, fully quit via the tray and relaunch.

17. Appendix — Route Reference

Page	Route	Who
Dashboard	/dashboard	All
Leads	/leads	Leads dept
All Leads (list)	/leads/list	Leads dept
Add Lead	/leads/new	Leads dept
Scraping	/scraping	Leads dept
Reports	/reports	boss/admin
Kanban	/kanban	Marketing
Calendar	/calendar	Marketing
Bug Reports	/feedback	All
Marketing Overview	/marketing	Marketing
Notes / Scripts	/marketing/notes · /marketing/scripts	Marketing
Badges / Challenges / Commissions	/marketing/manage/*	Marketing / boss
Balloon Pop	/balloons	Marketing
Users / Fleet	/admin/users · /admin/fleet	boss/admin
My Profile	/profile	All
Settings	/settings	boss (full)
How It Works	/how-it-works	boss/admin

— End of Standard Operating Procedure · DataForge v1.0 —